CS010A Course Slides



Scan to access all CS010A course slides

Week 5 Lab - CS010A

Password Validation

TA: Sourabha Bharadwaj
Instructors: Prof. Kelly Downey and Saul Gonzalez

Recap: Multiple `if` Statements

Is our superhero ready for battle?

```
int energy = 40;
bool hasWeapon = false;
bool ready = true;

if (energy < 50) {
    cout << "Not enough energy!" << endl;
    ready = false;
}
if (!hasWeapon) {
    cout << "You need a weapon!" << endl;
    ready = false;
}
if (ready) {
    cout << "Go save the world!" << endl;
}
```

Recap: Output and Discussion

Code

```
int energy = 40;
bool hasWeapon = false;
bool ready = true;

if (energy < 50) {
    cout << "Not enough energy!" << endl;
    ready = false;
}
if (!hasWeapon) {
    cout << "You need a weapon!" << endl;
    ready = false;
}
if (ready) {
    cout << "Go save the world!" << endl;
}
```

Output

```
Not enough energy!
You need a weapon!
```

Discuss

- Why did both messages print?
- What if we used `else if`?
- When will `ready` stay `true`?

Recap: `for` Loops and Strings

A loop can visit every character in a string.

```
string name = "Flash";

for (int i = 0; i < name.size(); i++) {
    cout << i << ": " << name[i] << endl;
}
```

Output

```
0: F
1: l
2: a
3: s
4: h
```

Remember

- Indexing begins at 0.
- `name.size()` gives the length.
- `name[i]` accesses one character.

The <cctype> Library

Useful functions for checking individual characters.

```
#include <cctype>
```

Function	Checks whether...	Example
isdigit(c)	c is a digit	'7': true
isalpha(c)	c is a letter	'A': true
islower(c)	c is lowercase	'a': true
isupper(c)	c is uppercase	'Z': true

```
char c = 'Q';
if (isalpha(c)) {
    cout << "It is a letter!" << endl;
}
```

Today's Problem: Password Validation

Input: One password string.

Check all four requirements:

Requirement	Error message
At least 8 characters	Too short
At least one letter	Missing letter
At least 1 and at most 2 digits	Wrong number of digits
At least one of `!`, `#`, `%`	Missing special

Print **every applicable error** in the order listed above. If there are no errors, print `OK`.

Sample Input and Output

Example 1

Input:

```
Hello
```

Output:

```
Too short
Wrong number of digits
Missing special
```

Example 2

Input:

```
Hello12%
```

Output:

```
OK
```

Question: Why can one password produce several error messages?

Your Turn

Work with your group, in this order:

1. **Read the requirements.** Identify inputs and outputs.
2. **Write an algorithm.** Use plain English with clear, descriptive steps.
3. **Design test cases.** Include valid and invalid passwords. Then dry run your tests.
4. **Write the C++ program.** Then run your tests.

Test cases come before code.